Notes:

- HW6 due Nov 27 (All github classroom)
- HW7 due Dec 4 (All gradescope)
- Final project reports due Dec 4
- Email me before today's lecture if you want to volunteer to present on Dec. 2. Otherwise, I will randomly assign students to each day's presentation slots (see Brightspace announcement)

Last class: Scipy functionality:

- interpolation, `interp1d`, `PPoly`, `RegularGridInterpolator`
- integration, `quad`, `fixed_quad`, `dblquad`, `nquad`, `solve_ivp`
- optimization, `minimize` and BFGS

In [1]:
```python
import scipy.optimize as optim
from rosenbrock import *

x0= np.zeros(5)
out = optim.minimize(rosenbrock, x0)
print(out)
```
```
  message: Optimization terminated successfully.
  success: True
   status: 0
      fun: 5.370412703770336e-11
        x: [ 1.000e+00  1.000e+00  1.000e+00  1.000e+00  1.000e+00]
      nit: 39
      jac: [ 3.001e-06 -4.512e-06  5.418e-06 -5.464e-06  1.709e-06]
 hess_inv: [[ 7.580e-03  1.260e-02 ...  4.744e-02  9.476e-02]
            [ 1.260e-02  2.518e-02 ...  9.478e-02  1.892e-01]
            ...
            [ 4.744e-02  9.478e-02 ...  3.805e-01  7.597e-01]
            [ 9.476e-02  1.892e-01 ...  7.597e-01  1.522e+00]]
     nfev: 294
     njev: 49
```

In [2]:
```python
np.ones(5)-out.x
```
Out[2]:
```
array([8.01802492e-07, 1.60017682e-06, 3.17669430e-06, 6.35231579e-06,
       1.27034988e-05])
```

`minimize` provides many other methods to do the minimization, some of which require or can use more information about $f(\vec{x})$.

To start, can supply ∇f, also called the Jacobian of f, to `minimize` with BFGS:

In [3]:
```python
out = optim.minimize(rosenbrock, x0, jac=drosenbrock)
print(out)
```
```
  message: Optimization terminated successfully.
  success: True
   status: 0
      fun: 1.308434732257813e-11
        x: [ 1.000e+00  1.000e+00  1.000e+00  1.000e+00  1.000e+00]
      nit: 40
      jac: [-1.035e-06  4.088e-06 -1.647e-06 -1.937e-06  8.771e-07]
 hess_inv: [[ 7.494e-03  1.218e-02 ...  4.647e-02  9.274e-02]
            [ 1.218e-02  2.474e-02 ...  9.331e-02  1.864e-01]
            ...
            [ 4.647e-02  9.331e-02 ...  3.777e-01  7.541e-01]
            [ 9.274e-02  1.864e-01 ...  7.541e-01  1.510e+00]]
     nfev: 49
     njev: 49
```

- 39 iterations using BFGS and approximating the Jacobian increases to 40, but we do many fewer functions evaluations in total, evaluating $f(\vec{x})$ and ∇f each 49 times

In [4]:
```python
np.ones(5)-out.x
```
Out[4]:
```
array([4.44318216e-09, 6.32169028e-09, 2.16122479e-08, 4.37004095e-08,
       8.30151999e-08])
```

- Get slightly more accurate solution using Jacobian

Can do direct Newton's method by provide f and ∇f and using finite differencing to approximation the Hessian

In [5]:
```python
out = optim.minimize(rosenbrock, x0, jac=drosenbrock, method="Newton-CG")
print(out)
```
```
 message: Optimization terminated successfully.
 success: True
  status: 0
     fun: 1.7981803564111244e-12
       x: [ 1.000e+00  1.000e+00  1.000e+00  1.000e+00  1.000e+00]
     nit: 42
     jac: [ 2.727e-06  2.117e-05  3.833e-05  2.324e-04 -1.298e-04]
    nfev: 56
    njev: 264
    nhev: 0
```

In [6]:
```python
np.ones(5)-out.x
```
Out[6]:
```
array([1.44571321e-07, 2.89461624e-07, 5.80151301e-07, 1.16305094e-06,
       2.33185544e-06])
```

Takes 42 iterations, with 56 function evaluations and 264 Jacobian evaluations.

Doing lots of work to approximate Hessian, can also provide that from analytical calculation.

In [7]:
```python
out = optim.minimize(rosenbrock, x0, jac=drosenbrock, hess = Hrosenbrock, method="Newton-CG")
print(out)
```
```
 message: Optimization terminated successfully.
 success: True
  status: 0
     fun: 1.073803796516941e-06
       x: [ 9.999e-01  9.998e-01  9.996e-01  9.991e-01  9.982e-01]
     nit: 41
     jac: [ 1.442e-02 -4.182e-03 -2.680e-04 -5.189e-04 -1.246e-03]
    nfev: 55
    njev: 55
    nhev: 41
```

In [8]:
```python
np.ones(5)-out.x
```
Out[8]:
```
array([0.00011228, 0.0002246 , 0.00044932, 0.00089908, 0.00179971])
```

Here, we're doing 41 iterations, with 55 function and Jacobian evaluations, and 41 Hessian evaluations.

Solution quality is a little worse, could try to fix that up by adjusting tolerances...

These methods use the conjugate gradient algorithm to solve the linear system at each step,

$$H_k(\vec{x}_{k+1} - \vec{x}_k) = -\nabla f(\vec{x}_k)$$

Can also directly apply a variant of conjugate gradient to the minimization problem.

In [9]:
```python
out = optim.minimize(rosenbrock, x0, jac=drosenbrock, method="CG")
print(out)
```
```
 message: Optimization terminated successfully.
 success: True
  status: 0
     fun: 1.812035928013707e-14
       x: [ 1.000e+00  1.000e+00  1.000e+00  1.000e+00  1.000e+00]
     nit: 67
     jac: [-2.950e-06  4.914e-06 -4.363e-06  2.539e-06 -6.130e-07]
    nfev: 135
    njev: 135
```

In [10]:
```python
np.ones(5)-out.x
```
Out[10]:
```
array([ 2.62031585e-09, -2.12109508e-09,  4.35245628e-09,  2.11712847e-09,
        7.29918970e-09])
```

- 67 iterations with 135 function and Jacobian evluations, and a good quality solution.

Also several other unconstrained minimization algorithms that are useful in some settings.

Specialized approaches for minimizing a function of a single variable

In [11]:
```python
optim.minimize_scalar(lambda x: np.sin(x)**2, bracket=(2,4))
```
Out[11]:
```
  message: 
           Optimization terminated successfully;
           The returned value satisfies the termination criteria
           (using xtol = 1.48e-08 )
  success: True
      fun: 7.144481539848359e-22
        x: 3.1415926536165224
      nit: 10
     nfev: 13
```

`minimize_scalar` requires 2 points that "bracket" a minimizer, uses Brent's method to find a solution within this bracket

Trust region and other methods for constrained optimization: $\min f(\vec{x})$ subject to $\vec{c}_\ell \le \vec{c}(\vec{x}) \le \vec{c}_u$, where

- $\vec{c}(\vec{x})$ is a vector of length m (usually different from n, the length of $\vec{x}$,
- $\vec{c}_\ell$ and $\vec{c}_u$ are lower and upper bounds on the components of $\vec{c}(\vec{x})$

Can also impose bounds specifically on elements of $\vec{x}$

In [12]:
```python
bounds = optim.Bounds(np.zeros(5), np.array([1, 1, 1, 1, 0.95]))
out = optim.minimize(rosenbrock, x0, jac=drosenbrock, bounds=bounds)
print(out)
```
```
  message: CONVERGENCE: RELATIVE REDUCTION OF F <= FACTR*EPSMCH
  success: True
   status: 0
      fun: 0.0008506716490731248
        x: [ 9.968e-01  9.937e-01  9.873e-01  9.748e-01  9.500e-01]
      nit: 31
      jac: [-6.185e-06 -4.083e-05 -1.265e-05  1.823e-05 -3.453e-02]
     nfev: 43
     njev: 43
 hess_inv: <5x5 LbfgsInvHessProduct with dtype=float64>
```

Now see best solution has `x[4] = 0.95`, with `x[3]` approximately $\sqrt{0.95}$, and so on

So bounds have a strong impact on the quality of the solution that can be found.

Further specialized routines for nonlinear least squares, linear programming, mixed integer linear programming, and more

numpy and scipy also provide substantial support for linear algebra (`scipy.sparse`) and graph algorithms

We now have some packages available to us that can help us speed up our programs, by providing better data structures and specialized tools that take advantage of them.

How do we decide that we should be using these?

Get some idea of where performance is being lost by adding timers to our code, and this is often a great first step.

Consider using a Monte-Carlo algorithm to estimate the value of π.

Generate n random complex numbers with real and imaginary parts independently and uniform distributed between 0 and 1. This generates points with modulus between 0 and $\sqrt{2}$, inside a square in the complex plane.

By counting the number of these points inside the unit circle, with $|z| < 1$, we can estimate π, as $\pi/4$ is the ratio of the part of the unit square that is inside the unit circle to the area of the unit square itself.

This is what's coded up in `estpi.py`

3 functions:

- `random_complex_point` to generate a point in the unit square in the complex plane
- `inside` to check if such a point is inside the unit disc
- `estimate_pi` loops over n random points, checking if they are inside unit circle or not

Let's see how this works. It's a probabilistic sample, so should run it several times to see results:

In [14]:
```python
import estpi
for exponent in range(1,7):
    n = 10**exponent
    estimates = [estpi.estimate_pi(n) for _ in range(5)]
    print(f"{n =:<10,} {estimates}")
```
```
n =10         [3.2, 3.6, 3.6, 2.0, 3.6]
n =100        [3.32, 3.16, 3.04, 3.08, 3.08]
n =1,000      [3.08, 3.176, 3.18, 3.128, 3.084]
n =10,000     [3.1364, 3.14, 3.138, 3.1412, 3.1548]
n =100,000    [3.14588, 3.14568, 3.14392, 3.13656, 3.14032]
n =1,000,000  [3.144512, 3.14134, 3.141952, 3.139096, 3.143336]
```

See slow convergence - can prove error in this approximation is bounded like $\mathcal{O}(n^{-1/2})$

Also see that, as expected, the larger we make n, the longer the estimation takes

One way to quantify time is to just add a timer:

In [17]:
```python
from datetime import datetime
for exponent in range(1,8):
    n = 10**exponent
    start = datetime.now()
    estimates = [estpi.estimate_pi(n) for _ in range(5)]
    end = datetime.now()
    print(f"{n =:<10,} {estimates} in {(end-start).total_seconds()/5} seconds average time")
```
```
n =10         [2.8, 3.2, 3.2, 3.2, 4.0] in 7.6e-06 seconds average time
n =100        [3.24, 3.12, 3.16, 3.24, 3.08] in 3.68e-05 seconds average time
n =1,000      [3.176, 3.092, 3.248, 3.088, 3.128] in 0.0005136 seconds average time
n =10,000     [3.1608, 3.1476, 3.1724, 3.1236, 3.1432] in 0.0044302000000000005 seconds average time
n =100,000    [3.13744, 3.13644, 3.1472, 3.14236, 3.13996] in 0.0379004 seconds average time
n =1,000,000  [3.141204, 3.141564, 3.14042, 3.139564, 3.14056] in 0.3617394 seconds average time
n =10,000,000 [3.1417604, 3.1414164, 3.1417788, 3.14255, 3.1423924] in 3.7803421999999998 seconds average time
```

Clear linear scaling in time required, particularly as n gets larger, with about 0.37 seconds for n = 1 million, and 3.8 seconds average time for $n = 10$ million

This is informative. We want larger n to get better estimates, but cost grows with n. Doesn't tell us, however, where the time is going.

To get better insight, we could try to measure time inside individual functions, but this gets tricky because both `inside` and `random_complex_point` are very simple, low-cost functions that we call many times. Difficult to precisely accumulate time when running these sorts of cheap routines many times.

Instead of doing this manually, we can make use of a profiler that collects this information for us.

What is a profiler?

It is a program analysis tool, that aims to provide insight into performance.

3 main classes of profilers:

1. Event-based profilers follow the execution path, recording each call to a function and recording how much time is spent (cumulatively) in each call.
2. Statistical profilers interrupt the program at regular intervals, recording information about the call stack to sample what is running and how it was called.
3. Instrumentation-based profilers rely on the developer adding instructions to track profile information.

While there are situations in which instrumentation-based profilers are useful, we tend to default to event-based or statistical profilers instead

Python provides a very simple event-based profiler in `cProfile`. Can use it in Jupyter by importing it as a module

In [18]:
```python
import cProfile
cProfile.run('estpi.estimate_pi(1000000)')
```
```
         7000556 function calls (7000550 primitive calls) in 1.445 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        2    0.000    0.000    0.000    0.000 <frozen abc>:121(__subclasscheck__)
        4    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap>:1390(_handle_fromlist)
      2/1    0.000    0.000    1.219    1.219 <string>:1(<module>)
        1    0.000    0.000    0.101    0.101 asyncio.py:206(_handle_events)
        1    0.000    0.000    0.000    0.000 asyncio.py:231(add_callback)
        4    0.000    0.000    0.000    0.000 attrsettr.py:43(__getattr__)
        4    0.000    0.000    0.000    0.000 attrsettr.py:66(_get_attr_opt)
        1    0.000    0.000    0.000    0.000 base_events.py:1939(_add_callback)
        1    0.102    0.102    0.120    0.120 base_events.py:1954(_run_once)
        1    0.000    0.000    0.000    0.000 base_events.py:2052(get_debug)
        1    0.000    0.000    0.000    0.000 base_events.py:548(_check_closed)
        3    0.000    0.000    0.000    0.000 base_events.py:766(time)
        1    0.000    0.000    0.000    0.000 base_events.py:817(call_soon)
        1    0.000    0.000    0.000    0.000 base_events.py:846(_call_soon)
        1    0.000    0.000    0.000    0.000 contextlib.py:108(__init__)
        1    0.000    0.000    0.000    0.000 contextlib.py:136(__enter__)
        1    0.000    0.000    0.000    0.000 contextlib.py:145(__exit__)
        1    0.000    0.000    0.000    0.000 contextlib.py:303(helper)
        1    0.000    0.000    0.000    0.000 decorator.py:200(fix)
        1    0.000    0.000    0.013    0.013 decorator.py:232(fun)
       26    0.000    0.000    0.000    0.000 enum.py:1154(__new__)
       60    0.000    0.000    0.000    0.000 enum.py:1587(_get_value)
       14    0.000    0.000    0.000    0.000 enum.py:1594(__or__)
        6    0.000    0.000    0.000    0.000 enum.py:1605(__and__)
       26    0.000    0.000    0.000    0.000 enum.py:695(__call__)
  1000000    0.139    0.000    0.197    0.000 estpi.py:17(inside)
  1000000    0.342    0.000    0.875    0.000 estpi.py:24(random_complex_point)
        1    0.241    0.241    1.219    1.219 estpi.py:5(estimate_pi)
        1    0.000    0.000    0.000    0.000 events.py:36(__init__)
        2    0.000    0.000    0.116    0.058 events.py:87(_run)
        1    0.000    0.000    0.000    0.000 history.py:1008(_writeout_input_cache)
        1    0.000    0.000    0.000    0.000 history.py:1016(_writeout_output_cache)
        1    0.001    0.001    0.007    0.007 history.py:1024(writeout_cache)
        2    0.000    0.000    0.000    0.000 history.py:1065(hold)
        1    0.000    0.000    0.013    0.013 history.py:92(only_when_enabled)
        4    0.000    0.000    0.000    0.000 inspect.py:2781(name)
       10    0.000    0.000    0.000    0.000 inspect.py:2793(kind)
        1    0.000    0.000    0.000    0.000 inspect.py:2875(__init__)
        1    0.000    0.000    0.000    0.000 inspect.py:2883(args)
        1    0.000    0.000    0.000    0.000 inspect.py:2906(kwargs)
        1    0.000    0.000    0.000    0.000 inspect.py:2936(apply_defaults)
        4    0.000    0.000    0.000    0.000 inspect.py:3076(parameters)
        1    0.000    0.000    0.000    0.000 inspect.py:3122(_bind)
        1    0.000    0.000    0.000    0.000 inspect.py:3264(bind)
        1    0.000    0.000    0.006    0.006 ioloop.py:750(_run_callback)
        2    0.000    0.000    0.101    0.050 iostream.py:158(_handle_event)
        2    0.000    0.000    0.000    0.000 iostream.py:214(_is_master_process)
        2    0.000    0.000    0.000    0.000 iostream.py:217(_check_mp_mode)
        2    0.000    0.000    0.000    0.000 iostream.py:256(closed)
        2    0.007    0.003    0.151    0.076 iostream.py:277(<lambda>)
        2    0.007    0.004    0.081    0.040 iostream.py:279(_really_send)
        2    0.000    0.000    0.000    0.000 queue.py:115(empty)
        2    0.000    0.000    0.000    0.000 queue.py:267(_qsize)
  2000000    0.376    0.000    0.533    0.000 random.py:498(uniform)
        2    0.000    0.000    0.000    0.000 selector_events.py:740(_process_events)
        1    0.003    0.003    0.008    0.008 selectors.py:540(select)
       14    0.000    0.000    0.000    0.000 socket.py:623(send)
        2    0.008    0.004    0.044    0.022 socket.py:700(send_multipart)
        2    0.001    0.001    0.006    0.003 socket.py:771(recv_multipart)
        2    0.000    0.000    0.000    0.000 threading.py:303(__enter__)
        2    0.000    0.000    0.000    0.000 threading.py:306(__exit__)
        1    0.000    0.000    0.000    0.000 threading.py:312(_release_save)
        1    0.000    0.000    0.000    0.000 threading.py:315(_acquire_restore)
        1    0.000    0.000    0.000    0.000 threading.py:318(_is_owned)
        1    0.000    0.000    0.000    0.000 threading.py:631(clear)
        1    0.000    0.000    0.000    0.000 traitlets.py:1512(_notify_trait)
        1    0.000    0.000    0.000    0.000 traitlets.py:1523(notify_change)
        1    0.000    0.000    0.000    0.000 traitlets.py:1527(_notify_observers)
        2    0.000    0.000    0.000    0.000 traitlets.py:2304(validate)
        2    0.000    0.000    0.000    0.000 traitlets.py:3474(validate)
        2    0.000    0.000    0.000    0.000 traitlets.py:3486(validate_elements)
        2    0.000    0.000    0.000    0.000 traitlets.py:3624(validate_elements)
        2    0.000    0.000    0.000    0.000 traitlets.py:3631(set)
        4    0.000    0.000    0.000    0.000 traitlets.py:629(get)
        4    0.000    0.000    0.000    0.000 traitlets.py:676(__get__)
        2    0.000    0.000    0.000    0.000 traitlets.py:689(set)
        2    0.000    0.000    0.000    0.000 traitlets.py:708(__set__)
        2    0.000    0.000    0.000    0.000 traitlets.py:718(_validate)
        2    0.000    0.000    0.000    0.000 traitlets.py:727(_cross_validate)
        2    0.000    0.000    0.000    0.000 typing.py:1374(__instancecheck__)
        4    0.000    0.000    0.000    0.000 typing.py:1443(__hash__)
        2    0.000    0.000    0.000    0.000 typing.py:1665(__subclasscheck__)
       20    0.000    0.000    0.000    0.000 typing.py:2371(cast)
        6    0.000    0.000    0.000    0.000 typing.py:426(inner)
        4    0.000    0.000    0.000    0.000 zmqstream.py:528(receiving)
        2    0.000    0.000    0.000    0.000 zmqstream.py:532(sending)
        2    0.000    0.000    0.101    0.050 zmqstream.py:546(_run_callback)
        2    0.000    0.000    0.107    0.054 zmqstream.py:573(_handle_events)
        2    0.000    0.000    0.107    0.054 zmqstream.py:614(_handle_recv)
        2    0.000    0.000    0.000    0.000 zmqstream.py:653(_rebuild_io_state)
        2    0.000    0.000    0.000    0.000 zmqstream.py:676(_update_handler)
        1    0.000    0.000    0.006    0.006 zmqstream.py:684(<lambda>)
        2    0.000    0.000    0.000    0.000 {built-in method _abc._abc_subclasscheck}
        1    0.000    0.000    0.000    0.000 {built-in method _asyncio.get_running_loop}
        1    0.000    0.000    0.000    0.000 {built-in method _contextvars.copy_context}
        1    0.000    0.000    0.000    0.000 {built-in method _thread.allocate_lock}
  1000000    0.057    0.000    0.057    0.000 {built-in method builtins.abs}
      2/1    0.000    0.000    1.219    1.219 {built-in method builtins.exec}
        5    0.000    0.000    0.000    0.000 {built-in method builtins.getattr}
        9    0.000    0.000    0.000    0.000 {built-in method builtins.hasattr}
        4    0.000    0.000    0.000    0.000 {built-in method builtins.hash}
  131/127    0.000    0.000    0.000    0.000 {built-in method builtins.isinstance}
        2    0.000    0.000    0.000    0.000 {built-in method builtins.issubclass}
        2    0.000    0.000    0.000    0.000 {built-in method builtins.iter}
       10    0.000    0.000    0.000    0.000 {built-in method builtins.len}
        2    0.000    0.000    0.000    0.000 {built-in method builtins.max}
        8    0.000    0.000    0.000    0.000 {built-in method builtins.next}
        2    0.000    0.000    0.000    0.000 {built-in method posix.getpid}
        3    0.000    0.000    0.000    0.000 {built-in method time.monotonic}
        2    0.000    0.000    0.000    0.000 {method '__enter__' of '_thread.lock' objects}
        6    0.000    0.000    0.000    0.000 {method '__exit__' of '_thread.lock' objects}
        2    0.000    0.000    0.000    0.000 {method '__exit__' of 'sqlite3.Connection' objects}
        3    0.000    0.000    0.000    0.000 {method 'acquire' of '_thread.lock' objects}
        3    0.000    0.000    0.000    0.000 {method 'append' of 'collections.deque' objects}
        5    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
        1    0.000    0.000    0.000    0.000 {method 'control' of 'select.kqueue' objects}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
        1    0.001    0.001    0.006    0.006 {method 'execute' of 'sqlite3.Connection' objects}
        2    0.000    0.000    0.000    0.000 {method 'extend' of 'list' objects}
        5    0.000    0.000    0.000    0.000 {method 'get' of 'dict' objects}
        3    0.000    0.000    0.000    0.000 {method 'items' of 'mappingproxy' objects}
        4    0.000    0.000    0.000    0.000 {method 'popleft' of 'collections.deque' objects}
  2000000    0.157    0.000    0.157    0.000 {method 'random' of '_random.Random' objects}
        1    0.000    0.000    0.000    0.000 {method 'release' of '_thread.lock' objects}
        2    0.000    0.000    0.107    0.054 {method 'run' of '_contextvars.Context' objects}
        4    0.000    0.000    0.000    0.000 {method 'upper' of 'str' objects}
        1    0.000    0.000    0.000    0.000 {method 'values' of 'mappingproxy' objects}
```

Output is jumbled, not easy to parse. Needs a sort

In [19]:
```python
cProfile.run('estpi.estimate_pi(1000000)', sort='cumtime')
```
```
         7000555 function calls (7000550 primitive calls) in 1.355 seconds

   Ordered by: cumulative time

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      2/1    0.006    0.003    1.339    1.339 {built-in method builtins.exec}
        1    0.000    0.000    1.288    1.288 <string>:1(<module>)
        1    0.256    0.256    1.288    1.288 estpi.py:5(estimate_pi)
  1000000    0.343    0.000    0.876    0.000 estpi.py:24(random_complex_point)
  2000000    0.376    0.000    0.534    0.000 random.py:498(uniform)
  1000000    0.139    0.000    0.196    0.000 estpi.py:17(inside)
  2000000    0.157    0.000    0.157    0.000 {method 'random' of '_random.Random' objects}
  1000000    0.057    0.000    0.057    0.000 {built-in method builtins.abs}
        1    0.000    0.000    0.045    0.045 decorator.py:232(fun)
        1    0.007    0.007    0.045    0.045 history.py:92(only_when_enabled)
        1    0.000    0.000    0.038    0.038 history.py:1024(writeout_cache)
        1    0.006    0.006    0.032    0.032 history.py:1008(_writeout_input_cache)
        2    0.000    0.000    0.007    0.003 events.py:87(_run)
        2    0.000    0.000    0.007    0.003 {method 'run' of '_contextvars.Context' objects}
        2    0.000    0.000    0.007    0.003 zmqstream.py:573(_handle_events)
        1    0.000    0.000    0.007    0.007 asyncio.py:206(_handle_events)
        2    0.000    0.000    0.007    0.003 zmqstream.py:614(_handle_recv)
        2    0.006    0.003    0.007    0.003 zmqstream.py:546(_run_callback)
        1    0.001    0.001    0.006    0.006 base_events.py:1954(_run_once)
        2    0.000    0.000    0.000    0.000 iostream.py:158(_handle_event)
        2    0.000    0.000    0.000    0.000 iostream.py:277(<lambda>)
        2    0.000    0.000    0.000    0.000 iostream.py:279(_really_send)
        2    0.000    0.000    0.000    0.000 socket.py:700(send_multipart)
        4    0.000    0.000    0.000    0.000 attrsettr.py:43(__getattr__)
        4    0.000    0.000    0.000    0.000 attrsettr.py:66(_get_attr_opt)
        1    0.000    0.000    0.000    0.000 ioloop.py:750(_run_callback)
        1    0.000    0.000    0.000    0.000 zmqstream.py:684(<lambda>)
        2    0.000    0.000    0.000    0.000 zmqstream.py:653(_rebuild_io_state)
       14    0.000    0.000    0.000    0.000 socket.py:623(send)
        2    0.000    0.000    0.000    0.000 zmqstream.py:676(_update_handler)
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
        2    0.000    0.000    0.000    0.000 traitlets.py:708(__set__)
        2    0.000    0.000    0.000    0.000 traitlets.py:3631(set)
       14    0.000    0.000    0.000    0.000 enum.py:1594(__or__)
        2    0.000    0.000    0.000    0.000 traitlets.py:689(set)
        1    0.000    0.000    0.000    0.000 decorator.py:200(fix)
       60    0.000    0.000    0.000    0.000 enum.py:1587(_get_value)
  131/127    0.000    0.000    0.000    0.000 {built-in method builtins.isinstance}
        2    0.000    0.000    0.000    0.000 socket.py:771(recv_multipart)
        6    0.000    0.000    0.000    0.000 enum.py:1605(__and__)
        1    0.000    0.000    0.000    0.000 selectors.py:540(select)
        1    0.000    0.000    0.000    0.000 {method 'control' of 'select.kqueue' objects}
        2    0.000    0.000    0.000    0.000 traitlets.py:718(_validate)
       26    0.000    0.000    0.000    0.000 enum.py:695(__call__)
        1    0.000    0.000    0.000    0.000 inspect.py:3264(bind)
        1    0.000    0.000    0.000    0.000 inspect.py:3122(_bind)
        1    0.000    0.000    0.000    0.000 asyncio.py:231(add_callback)
        1    0.000    0.000    0.000    0.000 traitlets.py:1512(_notify_trait)
        2    0.000    0.000    0.000    0.000 typing.py:1374(__instancecheck__)
        6    0.000    0.000    0.000    0.000 typing.py:426(inner)
        1    0.000    0.000    0.000    0.000 base_events.py:817(call_soon)
        2    0.000    0.000    0.000    0.000 {method '__exit__' of 'sqlite3.Connection' objects}
        2    0.000    0.000    0.000    0.000 typing.py:1665(__subclasscheck__)
        2    0.000    0.000    0.000    0.000 traitlets.py:3474(validate)
        1    0.000    0.000    0.000    0.000 traitlets.py:1523(notify_change)
        1    0.000    0.000    0.000    0.000 base_events.py:846(_call_soon)
       26    0.000    0.000    0.000    0.000 enum.py:1154(__new__)
        4    0.000    0.000    0.000    0.000 traitlets.py:676(__get__)
        1    0.000    0.000    0.000    0.000 traitlets.py:1527(_notify_observers)
        1    0.000    0.000    0.000    0.000 {method 'execute' of 'sqlite3.Connection' objects}
        1    0.000    0.000    0.000    0.000 inspect.py:2936(apply_defaults)
        1    0.000    0.000    0.000    0.000 events.py:36(__init__)
        4    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap>:1390(_handle_fromlist)
        2    0.000    0.000    0.000    0.000 zmqstream.py:532(sending)
        2    0.000    0.000    0.000    0.000 {built-in method builtins.issubclass}
        2    0.000    0.000    0.000    0.000 selector_events.py:740(_process_events)
        1    0.000    0.000    0.000    0.000 contextlib.py:303(helper)
        2    0.000    0.000    0.000    0.000 traitlets.py:727(_cross_validate)
        8    0.000    0.000    0.000    0.000 {built-in method builtins.next}
        2    0.000    0.000    0.000    0.000 queue.py:115(empty)
        2    0.000    0.000    0.000    0.000 <frozen abc>:121(__subclasscheck__)
        3    0.000    0.000    0.000    0.000 base_events.py:766(time)
        9    0.000    0.000    0.000    0.000 {built-in method builtins.hasattr}
        1    0.000    0.000    0.000    0.000 base_events.py:1939(_add_callback)
        5    0.000    0.000    0.000    0.000 {built-in method builtins.getattr}
        3    0.000    0.000    0.000    0.000 {method 'items' of 'mappingproxy' objects}
        2    0.000    0.000    0.000    0.000 traitlets.py:3624(validate_elements)
        1    0.000    0.000    0.000    0.000 contextlib.py:145(__exit__)
        4    0.000    0.000    0.000    0.000 traitlets.py:629(get)
        1    0.000    0.000    0.000    0.000 threading.py:315(_acquire_restore)
        1    0.000    0.000    0.000    0.000 contextlib.py:108(__init__)
        1    0.000    0.000    0.000    0.000 contextlib.py:136(__enter__)
        1    0.000    0.000    0.000    0.000 threading.py:312(_release_save)
        2    0.000    0.000    0.000    0.000 {built-in method _abc._abc_subclasscheck}
        1    0.000    0.000    0.000    0.000 history.py:1016(_writeout_output_cache)
        2    0.000    0.000    0.000    0.000 iostream.py:217(_check_mp_mode)
        2    0.000    0.000    0.000    0.000 traitlets.py:2304(validate)
        1    0.000    0.000    0.000    0.000 inspect.py:2883(args)
        4    0.000    0.000    0.000    0.000 typing.py:1443(__hash__)
        1    0.000    0.000    0.000    0.000 threading.py:631(clear)
        2    0.000    0.000    0.000    0.000 iostream.py:214(_is_master_process)
        2    0.000    0.000    0.000    0.000 threading.py:306(__exit__)
       10    0.000    0.000    0.000    0.000 {built-in method builtins.len}
        1    0.000    0.000    0.000    0.000 {built-in method _thread.allocate_lock}
        1    0.000    0.000    0.000    0.000 {method 'values' of 'mappingproxy' objects}
        5    0.000    0.000    0.000    0.000 {method 'get' of 'dict' objects}
        2    0.000    0.000    0.000    0.000 threading.py:303(__enter__)
       20    0.000    0.000    0.000    0.000 typing.py:2371(cast)
        6    0.000    0.000    0.000    0.000 {method '__exit__' of '_thread.lock' objects}
        4    0.000    0.000    0.000    0.000 {method 'upper' of 'str' objects}
        2    0.000    0.000    0.000    0.000 {built-in method builtins.iter}
        1    0.000    0.000    0.000    0.000 inspect.py:2906(kwargs)
        5    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
        2    0.000    0.000    0.000    0.000 queue.py:267(_qsize)
        4    0.000    0.000    0.000    0.000 {method 'popleft' of 'collections.deque' objects}
        1    0.000    0.000    0.000    0.000 threading.py:318(_is_owned)
        2    0.000    0.000    0.000    0.000 traitlets.py:3486(validate_elements)
        3    0.000    0.000    0.000    0.000 {method 'acquire' of '_thread.lock' objects}
        2    0.000    0.000    0.000    0.000 {built-in method builtins.max}
       10    0.000    0.000    0.000    0.000 inspect.py:2793(kind)
        3    0.000    0.000    0.000    0.000 {method 'append' of 'collections.deque' objects}
        1    0.000    0.000    0.000    0.000 {built-in method _contextvars.copy_context}
        4    0.000    0.000    0.000    0.000 inspect.py:2781(name)
        2    0.000    0.000    0.000    0.000 history.py:1065(hold)
        4    0.000    0.000    0.000    0.000 zmqstream.py:528(receiving)
        1    0.000    0.000    0.000    0.000 {built-in method _asyncio.get_running_loop}
        2    0.000    0.000    0.000    0.000 {method 'extend' of 'list' objects}
        4    0.000    0.000    0.000    0.000 {built-in method builtins.hash}
        3    0.000    0.000    0.000    0.000 {built-in method time.monotonic}
        2    0.000    0.000    0.000    0.000 {built-in method posix.getpid}
        4    0.000    0.000    0.000    0.000 inspect.py:3076(parameters)
        2    0.000    0.000    0.000    0.000 {method '__enter__' of '_thread.lock' objects}
        1    0.000    0.000    0.000    0.000 base_events.py:2052(get_debug)
        2    0.000    0.000    0.000    0.000 iostream.py:256(closed)
        1    0.000    0.000    0.000    0.000 {method 'release' of '_thread.lock' objects}
        1    0.000    0.000    0.000    0.000 inspect.py:2875(__init__)
        1    0.000    0.000    0.000    0.000 base_events.py:548(_check_closed)
```

How do we read this:

- overall runtime of about 1.35 seconds.
- first 2 lines are just cProfile startup
- function `estimate_pi` took about 1.3 seconds in total, about 0.25 second by itself, and about 1 second in other functions.
- `cumtime` is total accumulated time in 1 function, `tottime` is time in that function *not including* time in functions called by that function

Of the 1.3 seconds in `estimate_pi`:

- about 0.9 seconds is spent in `random_complex_point`, which was called 1000000 times (as expected)
- about 0.5 seconds of that was spent in `random.uniform`
- another 0.2 seconds was spent in `inside`, of which 0.06 seconds was in `abs`

2 things to notice:

1. Hard to parse - no connections, just get output that relies on our ability to interpret it
2. Jump in time from our original check using `datetime`.

Profiling isn't free! Takes computational power to track the information, so running code with a profiler takes more time than running without.

Note, however, that lots of things impact total time of execution, including running in a Jupyter notebook. Can run `cProfile` at the command line:

```
(CMSC6950_venv) [math-scott02:CMSC6950/Fall2025/Notes] smaclachlan% python -m cProfile -s cumtime estpi.py --n 1000000
With 1000000 points, estimate of pi is 3.144536
         7005040 function calls (7004933 primitive calls) in 1.303 seconds

   Ordered by: cumulative time

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      9/1    0.000    0.000    1.303    1.303 {built-in method builtins.exec}
        1    0.000    0.000    1.303    1.303 estpi.py:1(<module>)
        1    0.247    0.247    1.295    1.295 estpi.py:5(estimate_pi)
  1000000    0.331    0.000    0.856    0.000 estpi.py:24(random_complex_point)
  2000000    0.371    0.000    0.525    0.000 random.py:498(uniform)
  1000000    0.138    0.000    0.193    0.000 estpi.py:17(inside)
  2000000    0.154    0.000    0.154    0.000 {method 'random' of '_random.Random' objects}
  1000000    0.055    0.000    0.055    0.000 {built-in method builtins.abs}
```

Not a huge difference in timing here, but can be large depending on Jupyter environment and what else you're running on your machine.

Nicer organization of similar, but less detailed information is given by `pyinstrument`, a statistical profiler:

In [21]:
```python
from pyinstrument import Profiler
with Profiler() as profiler:
    estpi.estimate_pi(n=1000000)
profiler.print()
```
```
  _     ._   __/__   _ _  _  _ _/_   Recorded: 11:32:35  Samples:  1263
 /_//_/// /_\ / //_// / //_'/ //     Duration: 1.263     CPU time: 1.257
/   _/                      v5.1.1

Profile at /var/folders/ym/d_bk4s6563gc8fw10symyfg00000gn/T/ipykernel_69620/3100454715.py:2

1.263 <module>  /var/folders/ym/d_bk4s6563gc8fw10symyfg00000gn/T/ipykernel_69620/3100454715.py:1
└─ 1.262 estimate_pi  estpi.py:5
   ├─ 0.856 random_complex_point  estpi.py:24
   │  ├─ 0.502 Random.uniform  random.py:498
   │  │  ├─ 0.357 [self]  random.py
   │  │  └─ 0.145 Random.random  <built-in>
   │  └─ 0.354 [self]  estpi.py
   ├─ 0.242 [self]  estpi.py
   └─ 0.164 inside  estpi.py:17
      ├─ 0.107 [self]  estpi.py
      └─ 0.057 abs  <built-in>
```

Notice `with Profiler() as profiler:` construction - this starts and stops profiling around the line with the `with` statement

Timings are similar to those recorded by `cProfile`, but presentation now shows nesting of call sequence, with times sorted from largest to smallest within each call. This is often very helpful.

Notice header says 1263 samples in 1.263 seconds of CPU time, so sampling every 0.001 seconds

Can control sampling frequency with `interval` kwarg to `profiler`:

In [22]:
```python
from pyinstrument import Profiler
with Profiler(interval=0.01) as profiler:
    estpi.estimate_pi(n=1000000)
profiler.print()
```
```
  _     ._   __/__   _ _  _  _ _/_   Recorded: 11:36:25  Samples:  124
 /_//_/// /_\ / //_// / //_'/ //     Duration: 1.243     CPU time: 1.241
/   _/                      v5.1.1

Profile at /var/folders/ym/d_bk4s6563gc8fw10symyfg00000gn/T/ipykernel_69620/2413360049.py:2

1.24 <module>  /var/folders/ym/d_bk4s6563gc8fw10symyfg00000gn/T/ipykernel_69620/2413360049.py:1
└─ 1.24 estimate_pi  estpi.py:5
   ├─ 0.79 random_complex_point  estpi.py:24
   │  ├─ 0.49 Random.uniform  random.py:498
   │  │  ├─ 0.31 [self]  random.py
   │  │  └─ 0.18 Random.random  <built-in>
   │  └─ 0.30 [self]  estpi.py
   ├─ 0.23 [self]  estpi.py
   └─ 0.22 inside  estpi.py:17
      ├─ 0.16 [self]  estpi.py
      └─ 0.06 abs  <built-in>
```

Just controls frequency of statistical sampling. Sometimes useful to control this, especially when runtimes are very short, so taking samples every 0.001 seconds doesn't do a good job of sampling performance. But generally best to profile with runs that take seconds or longer to get useful data.

Also sometimes useful to compare profiles for runs with 2 different sizes, to see if any component part scales in an unexpected way. Here:

In [23]:
```python
from pyinstrument import Profiler
with Profiler(interval=0.01) as profiler:
    estpi.estimate_pi(n=10000000)
profiler.print()
```
```
  _     ._   __/__   _ _  _  _ _/_   Recorded: 11:38:28  Samples:  1213
 /_//_/// /_\ / //_// / //_'/ //     Duration: 12.137    CPU time: 12.115
/   _/                      v5.1.1

Profile at /var/folders/ym/d_bk4s6563gc8fw10symyfg00000gn/T/ipykernel_69620/1937216721.py:2

12.13 <module>  /var/folders/ym/d_bk4s6563gc8fw10symyfg00000gn/T/ipykernel_69620/1937216721.py:1
└─ 12.13 estimate_pi  estpi.py:5
   ├─ 8.12 random_complex_point  estpi.py:24
   │  ├─ 4.66 Random.uniform  random.py:498
   │  │  ├─ 3.26 [self]  random.py
   │  │  └─ 1.40 Random.random  <built-in>
   │  └─ 3.46 [self]  estpi.py
   ├─ 2.23 [self]  estpi.py
   └─ 1.78 inside  estpi.py:17
      ├─ 1.17 [self]  estpi.py
      └─ 0.61 abs  <built-in>
```

Bigger times here, but still in same ratio.

How do we make it faster?